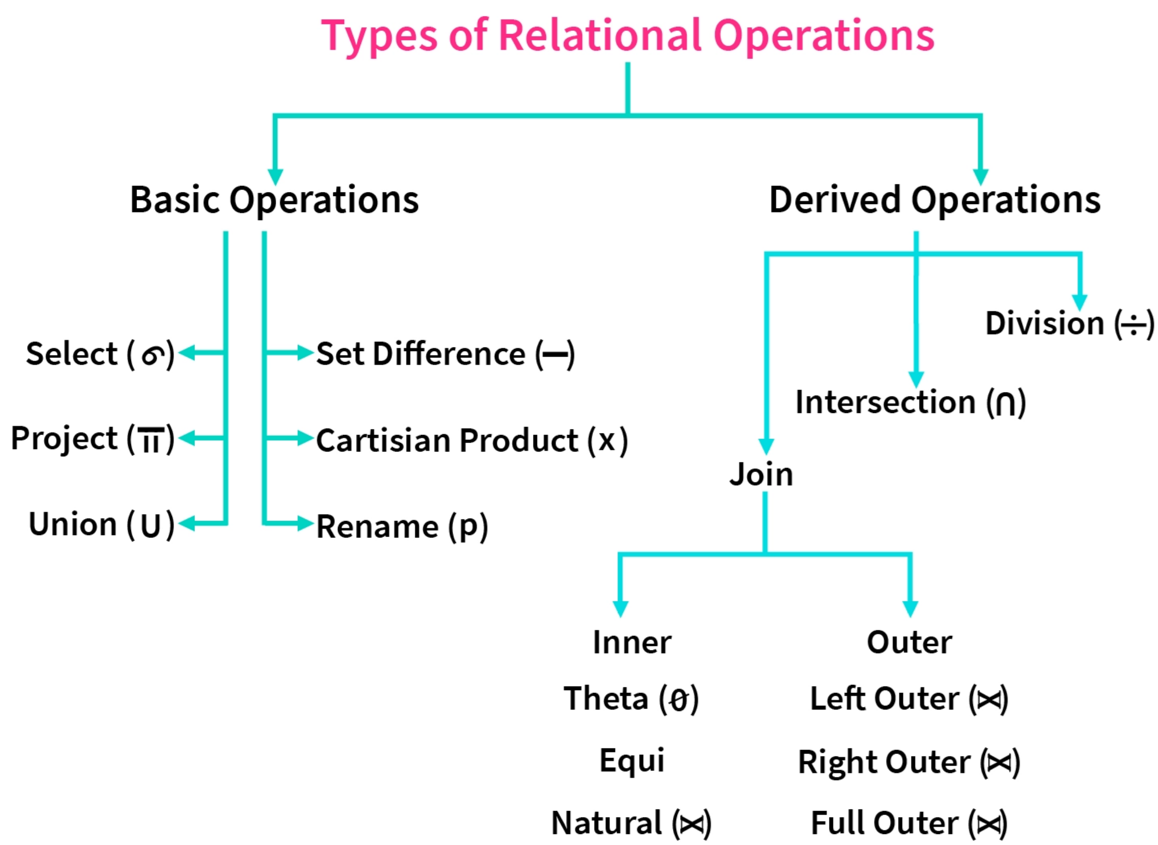
Relational Algebra is a procedural query language. Relational algebra mainly provides a theoretical foundation for relational databases and SQL. The main purpose of using Relational Algebra is to define operators that transform one or more input relations into an output relation. Given that these operators accept relations as input and produce relations as output, they can be combined and used to express complex queries that transform multiple input relations (whose data are stored in the database) into a single output relation (the query results).



STUDENT

ROLL	NAME	AGE
1	Aman	20
2	Atul	18
3	Baljeet	19
4	Harsh	20
5	Prateek	21
6	Prateek	23

EMPLOYEE

EMPLOYEE_NO	NAME	AGE
E-1	Anant	20
E-2	Ashish	23
E-3	Baljeet	25
E-4	Harsh	20
E-5	Pranav	22

Select (σ)

Select operation is done by Selection Operator which is represented by "sigma"(σ). It is used to retrieve tuples(rows) from the table where the given condition is satisfied. It is a **unary operator** means it requires only one operand.

Notation : σ p(R)
Where σ is used to represent SELECTION
R is used to represent RELATION
p is the logic formula
Let's understand this with an example:
Suppose we want the row(s) from STUDENT Relation where "AGE" is 20

σ AGE=20 (STUDENT)

This will return the following output:

ROLL	NAME	AGE
1	Aman	20
4	Harsh	20

Project (∏)

Project operation is done by Projection Operator which is represented by "pi"(∏). It is used to retrieve certain attributes(columns) from the table. It is also known as vertical partitioning as it separates the table vertically. It is also a **unary operator**.

Notation : ∏ a(r)
Where ∏ is used to represent PROJECTION
r is used to represent RELATION
a is the attribute list

Let's understand this with an example:
Suppose we want the names of all students from STUDENT Relation.

∏ NAME(STUDENT)

This will return the following output:

NAME
Aman
Atul
Baljeet
Harsh
Prateek

As you can see from the above output it **eliminates duplicates**.
For multiple attributes, we can separate them using a ",".

∏ ROLL,NAME(STUDENT)

Above code will return two columns, ROLL and NAME.

ROLL	NAME
1	Aman
2	Atul
3	Baljeet
4	Harsh
5	Prateek
6	Prateek

Cartesian product

The Cartesian product is used to combine each row in one table with each row in the other table. It is also known as a cross product.

- It is denoted by X.
- Notation: R X S
  Where R is the first relation
  S is the second relation
- As we can see from the notation it is also a **binary operator**.
- Let's combine the two relations STUDENT and EMPLOYEE.

Example:

EMPLOYEE

EMP_ID	EMP_NAME	EMP_DEPT
1	Smith	A
2	Harry	C
3	John	B

DEPARTMENT

DEPT_NO	DEPT_NAME
A	Marketing
B	Sales
C	Legal

Input:

1. EMPLOYEE X DEPARTMENT

Output:

EMP_ID	EMP_NAME	EMP_DEPT	DEPT_NO	DEPT_NAME
1	Smith	A	A	Marketing
1	Smith	A	B	Sales
1	Smith	A	C	Legal
2	Harry	C	A	Marketing
2	Harry	C	B	Sales

2	Harry	C	C	Legal
3	John	B	A	Marketing
3	John	B	B	Sales
3	John	B	C	Legal

Union operation is done by Union Operator which is represented by "union"(∪). It is the same as the union operator from set theory, i.e., **it selects all tuples from both relations but with the exception that for the union of two relations/tables both relations must have the same set of Attributes.** It is a **binary operator** as it requires two operands.
Notation: R ∪ S
Where R is the first relation
S is the second relation
If relations don't have the same set of attributes, then the union of such relations will result in NULL.
Let's have an example to clarify the concept:
Suppose we want all the names from STUDENT and EMPLOYEE relation.

∏ NAME(STUDENT) ∪ ∏ NAME(EMPLOYEE)

NAME
Aman
Anant
Ashish
Atul
Baljeet
Harsh
Pranav
Prateek

As we can see from the above output it also **eliminates duplicates**

Set Difference (-)

Set Difference as its name indicates is the difference between two relations (R-S). It is denoted by a "Hyphen"(-) and it returns all the tuples(rows) which are in relation R but not in relation S. It is also a binary operator.

Notation : R - S

Where R is the first relation

S is the second relation

Just like union, the set difference also comes with the exception of the same set of attributes in both relations.

Let's take an example where we would like to know the names of students who are in STUDENT Relation but not in EMPLOYEE Relation.

∏ NAME(STUDENT) - ∏ NAME(EMPLOYEE)

This will give us the following output:

NAME
Aman
Atul
Prateek